

Documento de Arquitectura de la segunda entrega del proyecto



Universidad
del Cauca

Vigilada Mineducación

INGENIERIA DE SOFTWARE 2

Presentado por:

Juan Camilo Henao Villegas

Juan Pablo Collazos Samboni

Santiago Majé Bonilla

Daniel Alexander Chaguendo

Profesor:

ROBERTO ENCARNACION MOSQUERA

Universidad del Cauca

Facultad de Ingeniería Electrónica y Telecomunicaciones

Ingeniería en Sistemas

Popayán, Mayo del 2026

1. Introducción	4
2. Historias de usuario y criterios de aceptación	1
3. Prototipos de interfaz de usuario y thinking aloud	2
4. Planificación de tareas en Jira	3
5. Escenario de calidad modificabilidad	3
6. Escenario de calidad escalabilidad	5
7. Arquitectura y diseño de software	6
8. Patrones de diseño implementados	10
8.0. Builder.....	10
8.1. Observer	11
8.2. State	11
8.3. Template Method	11
8.4. Singleton.....	12
8.5. Factory Method	12
8.6. Strategy (Comportamental).....	12

CONTENIDO

Índice de ilustraciones

No se encuentran elementos de tabla de ilustraciones.

Índice de tablas

Tabla 1. Escenario 1: Recuperación ante falla del servidor durante horario de agendamiento.	6
Tabla 2. Escenario 2 :Manejo de carga concurrente.	7
Tabla 3. Escenario 3: Intento de acceso no autorizado a historia clínica	8
Tabla 4. Escenario 4: Ataque de fuerza bruta al sistema de autenticación	9

1. Introducción

El presente documento describe la arquitectura de software del sistema Piedrazul, una plataforma de gestión de citas médicas desarrollada con una arquitectura de microservicios. El sistema permite a pacientes, profesionales de salud, administradores y agendadores interactuar con la plataforma a través de una aplicación de escritorio construida con JavaFX, la cual se comunica con el backend exclusivamente mediante una API REST.

El backend está compuesto por tres microservicios independientes MS Auth, MS UserManagement y MS Scheduling coordinados a través de un API Gateway centralizado y un bus de mensajería asíncrona basado en RabbitMQ. Cada microservicio gestiona su propia base de datos PostgreSQL, garantizando el aislamiento de datos y la independencia de despliegue.

Para documentar la arquitectura se emplea el modelo C4, que permite describir el sistema en cuatro niveles de detalle progresivo: contexto, contenedores, componentes y código. Este modelo se complementa con diagramas UML de clases, secuencia, estados y despliegue, que ilustran los patrones de diseño aplicados Builder, State, Template Method y Observer así como los flujos de interacción más relevantes del sistema.

2. Historias de usuario y criterios de aceptación

		Enunciado de la historia				Criterios de aceptación			
Identificador (ID) de la historia	Identificador (ID) de la historia	Rol	Característica/ Funcionalidad (Quiero)	Razón / Resultado (Para)	Número (#) de escenario	Criterio de aceptación (Título)	Contexto	Evento	Resultado / Comportamiento esperado
			Quiero consultar las citas de un médico o terapeuta	Para visualizar la programación diaria	1	Búsqueda de citas exitosa	El agendador selecciona médico/terapeuta y fecha	Clic en botón BUSCAR	El sistema muestra el listado de citas programadas y la cantidad total de citas
					2	No existen citas registradas	El agendador realiza la búsqueda	Clic en botón BUSCAR	El sistema muestra mensaje "No existen citas programadas para la fecha seleccionada"
					3	Validación de campos obligatorio	El agendador no selecciona médico o fecha	Clic en botón BUSCAR	El sistema muestra mensaje indicando que los campos son obligatorios
					4	Visualización de resultados	Existen citas registradas	Finaliza la búsqueda	El sistema muestra una tabla con paciente, hora y estado de la cita
	HU-6.1	Como Agendador de citas							
HU-6.2	Agendador de citas	Quiero registrar una nueva cita manualmente	Para programar citas solicitadas por WhatsApp o llamada	1	Registro exitoso de cita	El agendador diligencia correctamente la información	Clic en botón GUARDAR		El sistema registra la cita y muestra mensaje de confirmación
				2	El sistema registra la cita y muestra mensaje de confirmación	Faltan datos requeridos del paciente o cita	Clic en botón GUARDAR		El sistema muestra mensaje indicando los campos faltantes
				3	Validación de horario disponible	El horario ya se encuentra ocupado	Clic en botón GUARDAR		El sistema impide el registro y muestra advertencia
				4	Registro de datos opcionales	El paciente no ingresa fecha de nacimiento o correo	Clic en botón GUARDAR		El sistema permite continuar con el registro de la cita
				5	Validación de intervalo de tiempo	La cita incumple el intervalo configurado del médico	Clic en botón GUARDAR		El sistema informa que el horario no está disponible
	HU-6.3	Como Agendador de citas	Quiero modificar una cita programada	Para actualizar cambios solicitados por el paciente	1	Reprogramación exitosa	La cita existe y el horario está	Clic en ACTUALIZAR	El sistema guarda los cambios correctamente
					2	Horario no disponible	El nuevo horario ya fue ocupado	Clic en ACTUALIZAR	El sistema bloquea la modificación
					3	Validación de datos	Existen datos inválidos o incompletos	Clic en ACTUALIZAR	El sistema muestra mensajes de validación
					4	Notificación de cambios	La cita fue modificada	Finaliza actualización	El sistema notifica el cambio al paciente
	HU-6.4	Como Agendador de citas	Quiero cancelar una cita programada	Para liberar el horario y mantener actualizada la agenda	1	Cancelación exitosa	La cita existe	Clic en CANCELAR	El sistema cambia el estado de la cita
					2	Confirmación de cancelación	El agendador selecciona cancelar	Confirma operación	El sistema solicita confirmación antes de cancelar
					3	Notificación al paciente	La cita fue cancelada	Finaliza cancelación	El sistema envía notificación al paciente
					4	Restricción de cancelación	La cita ya fue atendida	Intenta cancelar	El sistema impide cancelar la cita

HE-06

3. Planificación de tareas en Jira

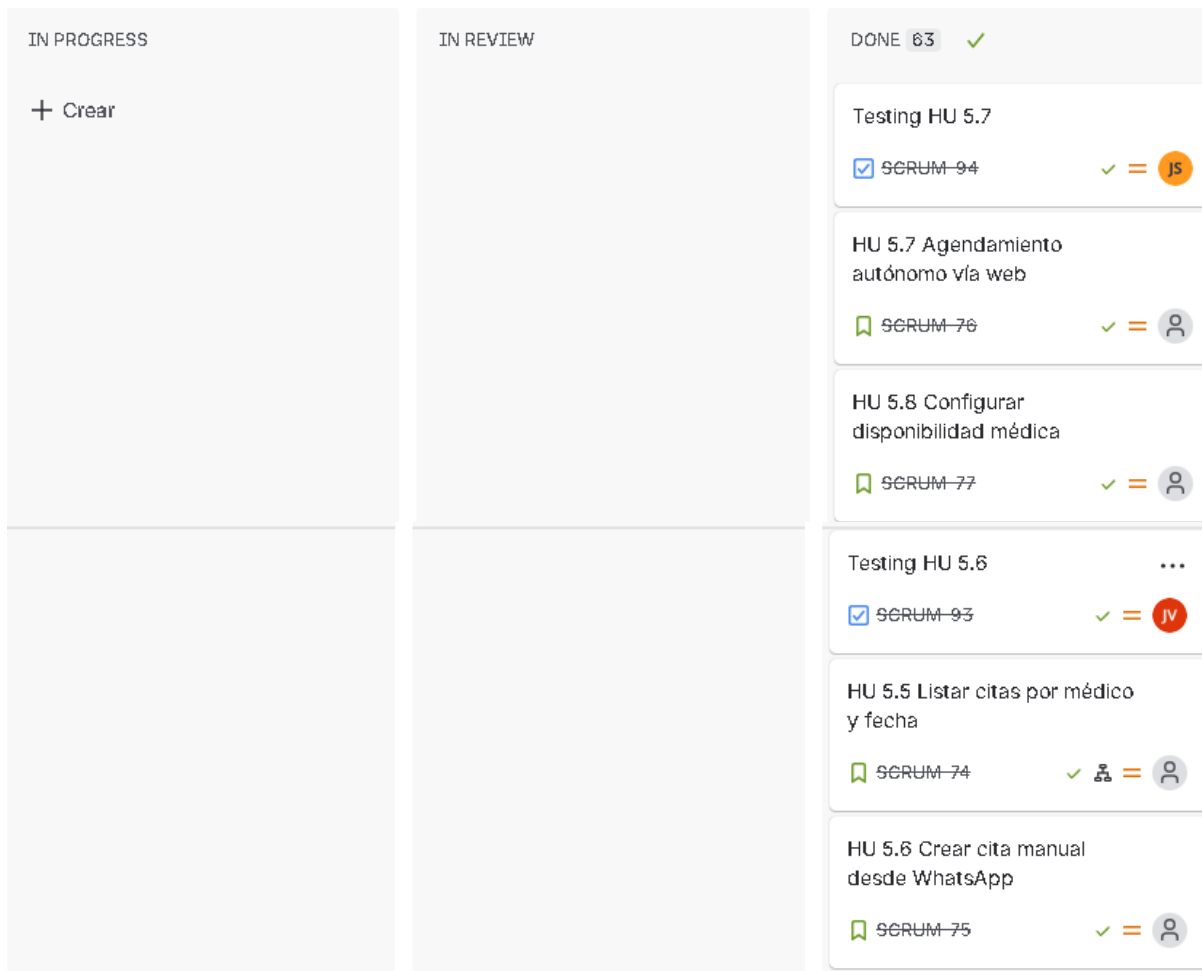


Imagen 4. Pantallazo de las tareas en Jira

4. Escenario de calidad modificabilidad

Porción del escenario	Descripción
Fuente	Equipo de desarrollo / Arquitecto de software

Estímulo	Se requiere refactorizar la aplicación monolítica para adoptar una arquitectura de microservicios basada en eventos, con el fin de mejorar la escalabilidad y mantenibilidad
Artefacto	Arquitectura del sistema (capas: presentación, negocio y persistencia) y módulos de dominio (citás, usuarios, configuración)
Entorno	Sistema en operación, con necesidad de evolución tecnológica sin interrupción del servicio
Respuesta	El sistema permite desacoplar progresivamente los módulos del monolito en servicios independientes, integrados mediante eventos (por ejemplo, publicación y suscripción), sin afectar el funcionamiento actual
Medida de respuesta	- Refactorización incremental por módulos (sin reescritura completa) ▪ Bajo acoplamiento entre componentes (facilitando su extracción como servicios) ▪ Introducción de un mecanismo de eventos sin impactar la lógica existente ▪ Disponibilidad del sistema $\geq 99\%$ durante la migración ▪ Tiempo de adaptación por módulo ≤ 1 semana Resultado esperado El sistema evoluciona hacia una arquitectura de microservicios orientada a eventos de manera gradual, reduciendo riesgos, manteniendo la continuidad del servicio y facilitando futuras extensiones

Justificación

Este escenario permite evaluar la capacidad del sistema para evolucionar desde una arquitectura monolítica hacia un enfoque de microservicios orientado a eventos, sin necesidad de rediseñar completamente la aplicación. Gracias a la separación por capas, el uso del patrón MVC, y la aplicación de principios SOLID, los módulos del sistema pueden desacoplarse progresivamente.

La introducción de comunicación basada en eventos (por ejemplo, mediante un broker de mensajería) permite reducir dependencias directas entre componentes, facilitando la escalabilidad y la incorporación de nuevos servicios en el futuro. Esto asegura que el sistema no solo sea modificable a nivel funcional, sino también a nivel arquitectónico.

5. Escenario de calidad escalabilidad

Fuente	Usuarios del sistema (pacientes, profesionales y agendadores)
Estímulo	Incremento masivo de solicitudes concurrentes de agendamiento y consulta de citas durante horarios pico
Artefacto	Microservicios MSAuth, MSUserManagement, MSScheduling, API Gateway y broker RabbitMQ
Entorno	Sistema en operación con alta demanda concurrente
Respuesta	El sistema distribuye la carga entre múltiples instancias de los microservicios, permitiendo procesar solicitudes simultáneamente sin degradar significativamente el rendimiento
Medida de respuesta	- Tiempo promedio de respuesta ≤ 2 segundos bajo 1000 solicitudes concurrentes. - Escalamiento horizontal independiente por microservicio \n- Disponibilidad del sistema $\geq 99\%$. - Uso de CPU por instancia $\leq 80\%$ durante picos de carga. Resultado esperado: El sistema soporta incrementos de carga mediante escalamiento horizontal de los microservicios, manteniendo disponibilidad, rendimiento y estabilidad

Justificación

La arquitectura basada en microservicios desacoplados permite escalar únicamente los servicios con mayor demanda, especialmente MSScheduling y MSAuth. El uso de API Gateway distribuye las solicitudes de entrada y RabbitMQ desacopla procesos asíncronos, evitando cuellos de botella. Esto facilita soportar crecimiento de usuarios y operaciones concurrentes sin afectar todo el sistema.

6. Arquitectura y diseño de software

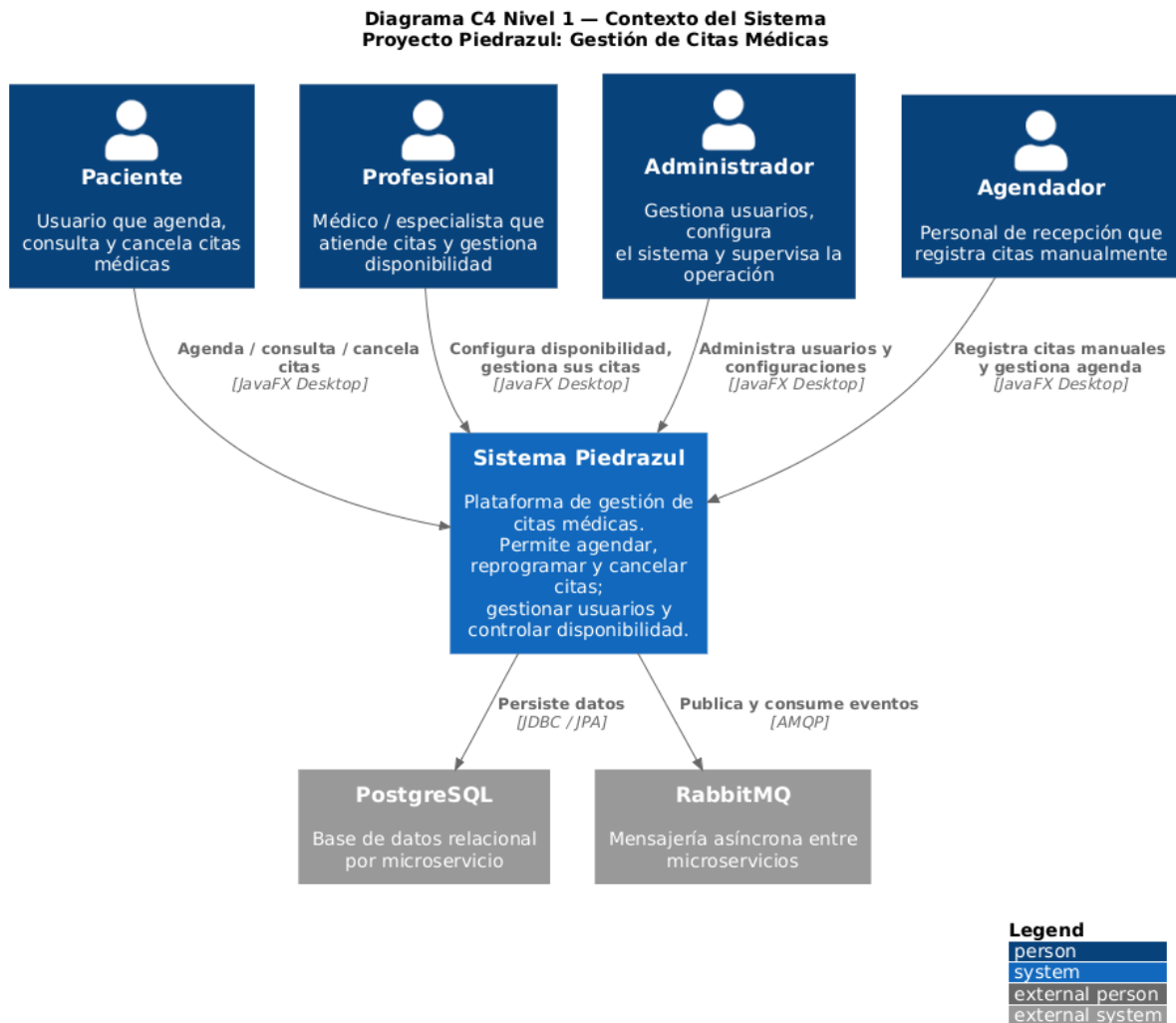


Imagen 5. Diagrama de contexto

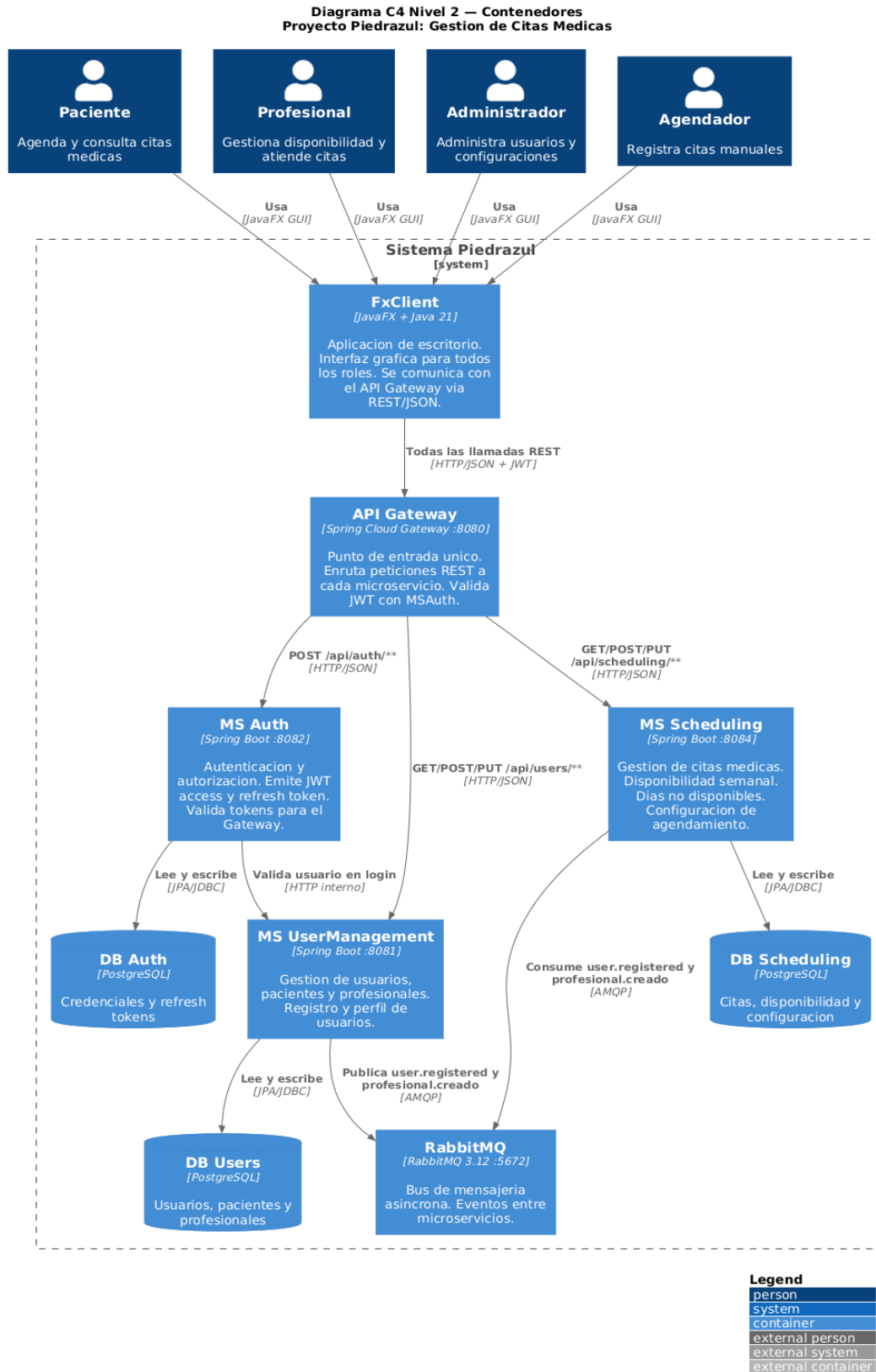


Imagen 6. Diagrama de contenedores

Bounded Context Diagram - Sistema Piedrazul

Arquitectura basada en microservicios

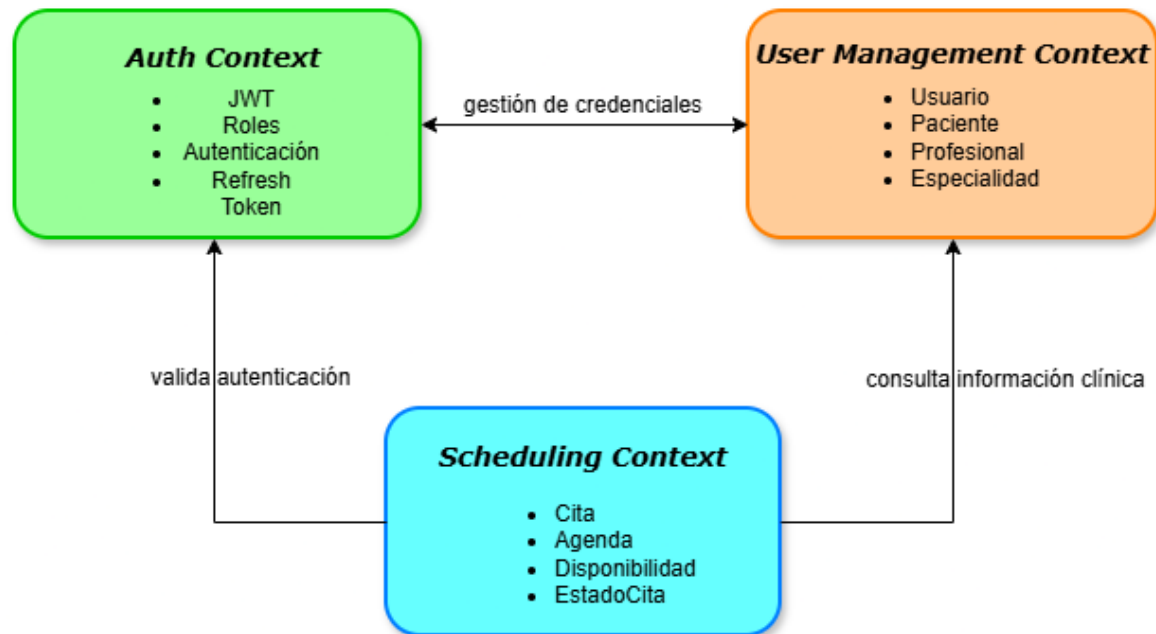


Imagen 7. Diagrama de contexto (Bounded Context).

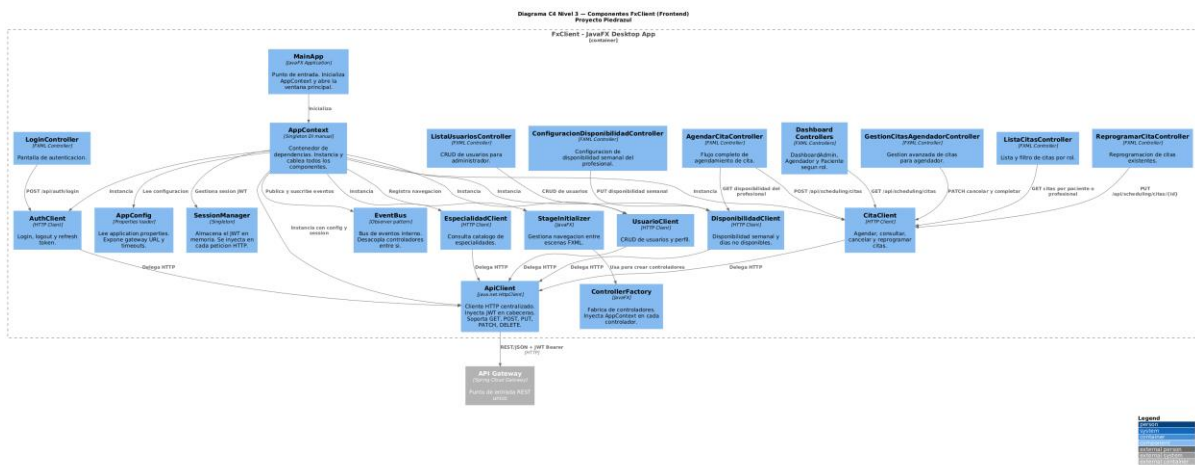


Imagen 8. Diagrama de componente FxCliente (Frontend)

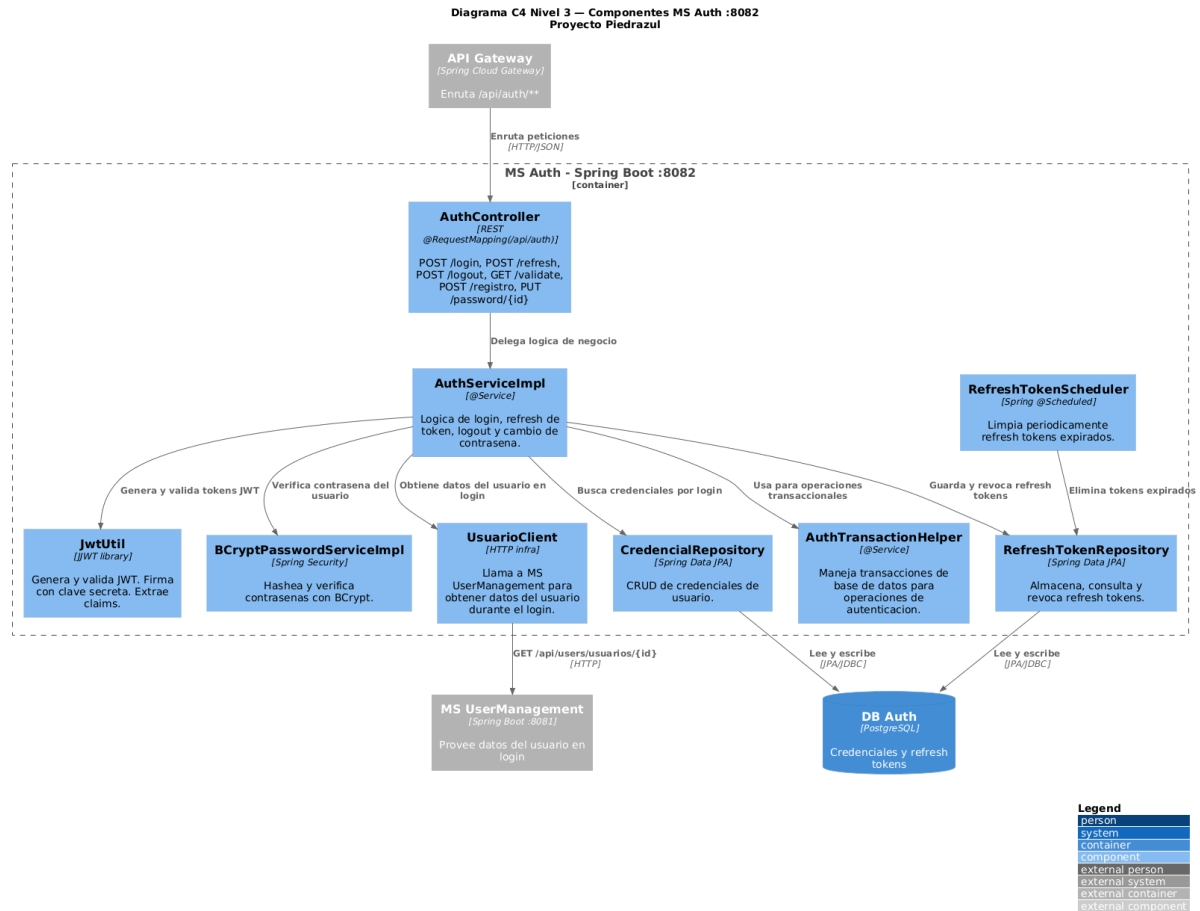


Imagen 9. Diagrama de componente MSAuth

Diagrama C4 Nivel 3 - Componentes MS UserManagement :8081
Proyecto Piedrazul

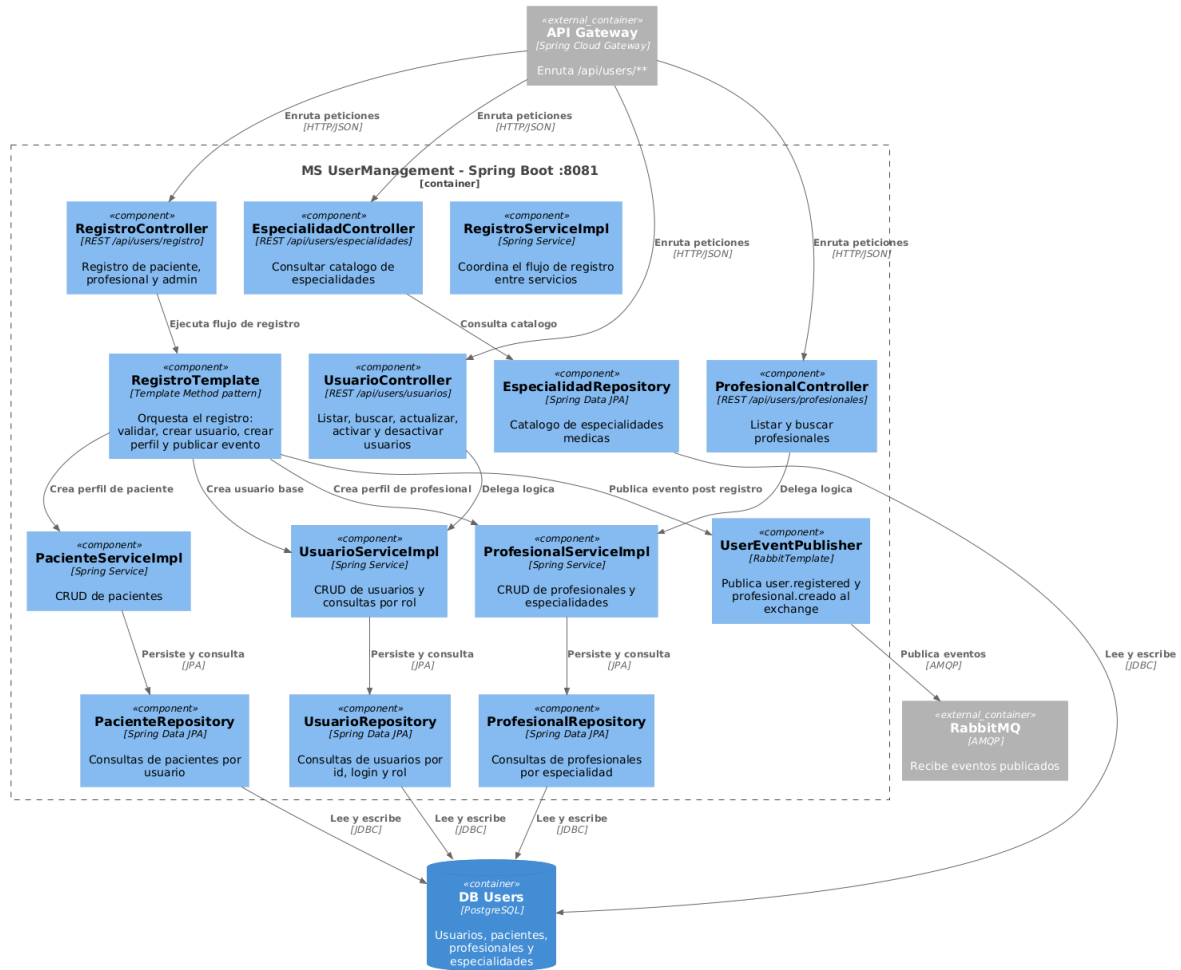


Imagen 10. Diagrama de componente MSUserManagement

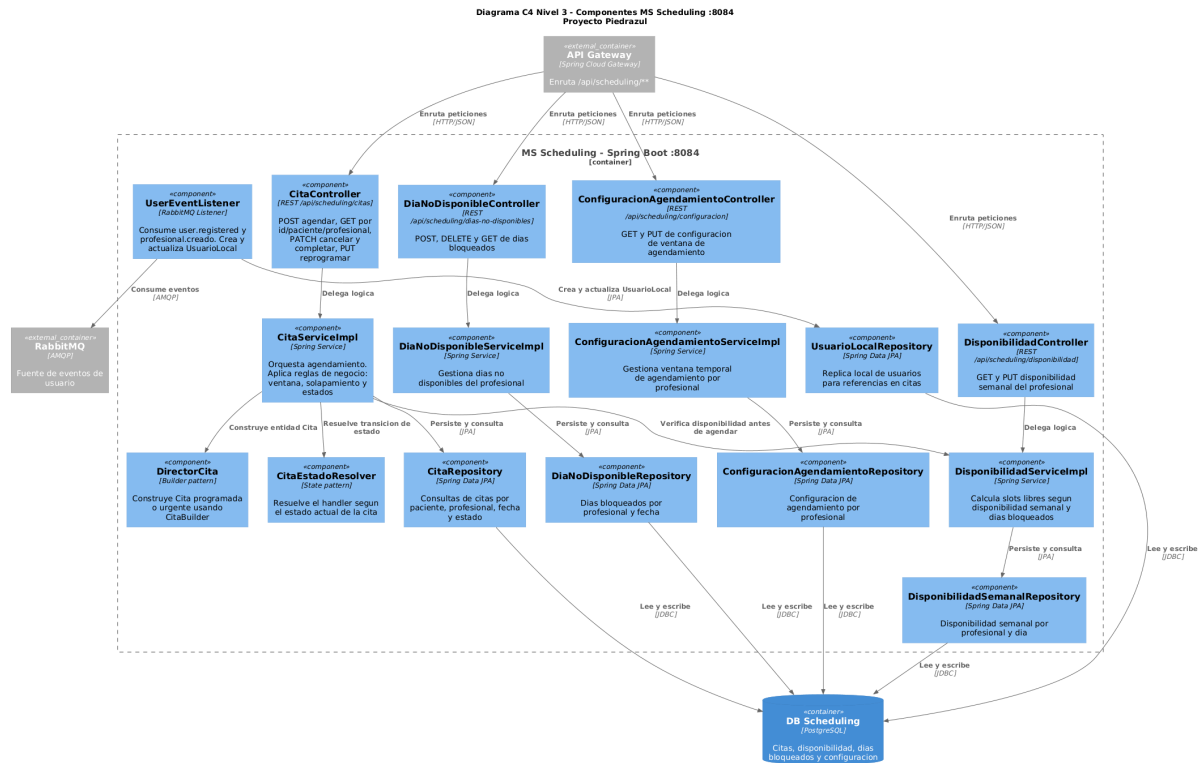


Imagen 11. Diagrama de componente MSSheduling

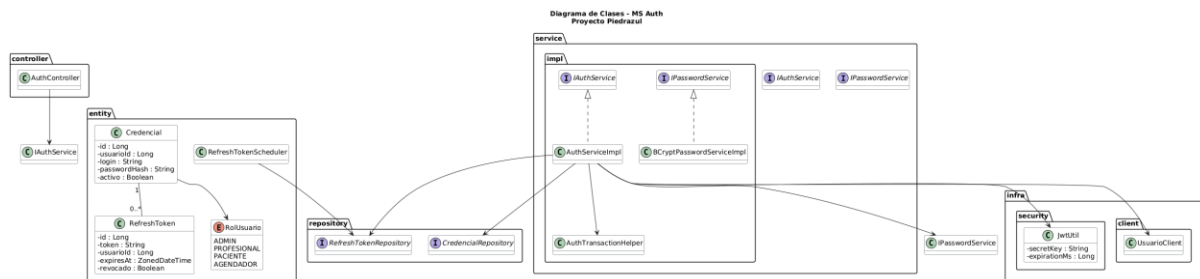


Imagen 12. Diagrama de clases de MS Auth

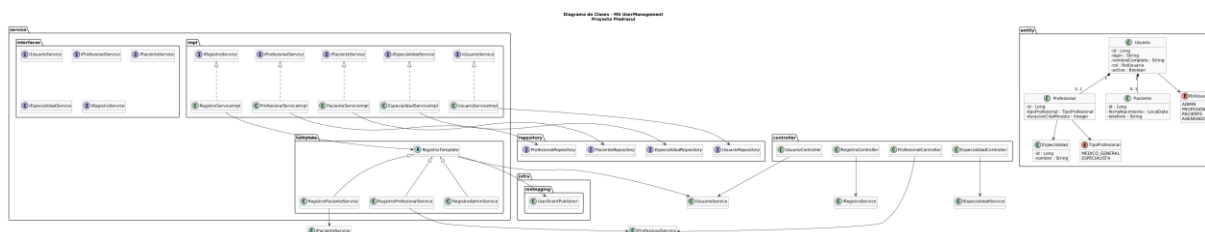


Imagen 13. Diagrama de clases de MS UserManagement

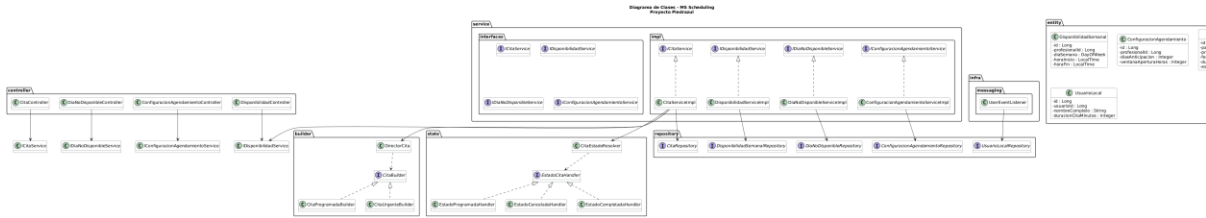


Imagen 13. Diagrama de clases de MS Scheduling

7. Patrones de diseño implementados

7.0. Builder

Ubicación: Monolito/model/builder/ y MSScheduling/domain/model/builder/

Clases: CitaBuilder (abstracta), CitaProgramadaBuilder, CitaUrgenteBuilder, DirectorCita

Cómo y por qué se aplicó: La creación de una Cita requiere ensamblar varios atributos obligatorios (paciente, profesional, fecha/hora, estado, fecha de creación). En lugar de tener un constructor con múltiples parámetros o lógica dispersa en el servicio, el patrón separa la construcción en pasos atómicos. El DirectorCita orquesta el orden de los pasos, mientras que las subclases concretas (CitaProgramadaBuilder, CitaUrgenteBuilder) varían el comportamiento específico de cada tipo de cita. En MSScheduling se añadió además el paso buildDuracion() para soportar ventanas de agendamiento configurables.

7.1. Observer

Ubicación: Monolito/observer/ y FxClient/observer/

Clases: Observer<T> (interfaz), EventBus, AppEvent (enum)

Cómo y por qué se aplicó: El sistema necesita que múltiples partes de la UI reaccionen a eventos del dominio (cita agendada, usuario creado, etc.) sin que los servicios conozcan directamente los controladores. El EventBus actúa como mediador: cualquier componente puede suscribirse a un AppEvent concreto, y cuando el servicio publica el evento, todos los observadores registrados son notificados. En el cliente JavaFX (FxClient) esto permitió desacoplar los controladores de pantalla de los clientes HTTP.

7.2. State

Ubicación: MSScheduling/domain/model/state/

Clases: EstadoCitaHandler (interfaz), EstadoProgramadaHandler, EstadoCanceladaHandler, EstadoCompletadaHandler, CitaEstadoResolver

Cómo y por qué se aplicó: El ciclo de vida de una cita tiene tres estados (programada, cancelada, completada) con reglas de transición distintas: solo desde programada se puede cancelar o completar; los otros dos son estados terminales. En

vez de acumular if/switch en el servicio, cada estado tiene su propia clase que encapsula qué transiciones permite y cuáles lanza `TransicionEstadoInvalidaException`. El `CitaEstadoResolver` usa inyección de Spring para construir automáticamente el mapa de despacho, respetando el principio Open/Closed.

7.3. Template Method

Ubicación: `MSUserManagement/application/service/template/`

Clases: `RegistroTemplate`(abstracta), `RegistroAdminService`, `RegistroPacienteService`, `RegistroProfesionalService`, `RegistroContexto`

Cómo y por qué se aplicó: Registrar un usuario siempre sigue tres pasos en el mismo orden: crear el usuario base, vincular el perfil específico, retornar el DTO. Solo el paso 2 varía según el tipo de usuario. `RegistroTemplate` define el esqueleto del algoritmo como método final y expone el hook `vincularPerfil()` para que cada subclase lo sobrescriba. El registro de administrador ni siquiera lo sobrescribe mientras que `RegistroPacienteService` y `RegistroProfesionalService` crean el perfil clínico correspondiente.

7.4. Singleton

Ubicación: `FxClient/http/SessionManager.java` y `FxClient/app/AppContext.java`

Cómo y por qué se aplicó: El cliente JavaFX no tiene contenedor Spring, por lo que se implementó manualmente el patrón Singleton para dos objetos de ciclo de vida global. `SessionManager` guarda el JWT y el usuario autenticado, garantizando que todos los controladores accedan al mismo token de sesión. `AppContext` actúa como contenedor de dependencias manual, instanciando y cableando todos los clientes HTTP una sola vez, reemplazando el `ApplicationContext` de Spring que sí existe en los microservicios.

7.5. Factory Method

Ubicación: `FxClient/app/ControllerFactory.java`

Cómo y por qué se aplicó: JavaFX necesita instanciar controladores FXML mediante un `Callback<Class<?>, Object>`. En el monolito esto lo hacía Spring automáticamente vía `context::getBean`. En el cliente desacoplado (`FxClient`), la `ControllerFactory` centraliza la creación de todos los controladores, inyectando manualmente sus dependencias desde el `AppContext`. Cada tipo de controlador tiene su propia rama de construcción, cumpliendo el rol de fábrica especializada sin exponer la lógica de ensamblado al resto de la aplicación.

7.6. Strategy (Comportamental)

Ubicación: `Monolito/model/services/interfaces/IPasswordService.java` y `MSAuth/application/service/IPasswordService.java`

Clases: IPasswordService (interfaz), BCryptPasswordServiceImpl

Cómo y por qué se aplicó: El algoritmo de encriptación de contraseñas se definió como una estrategia intercambiable mediante la interfaz IPasswordService. Los servicios que necesitan verificar o cifrar contraseñas dependen de la abstracción, no de BCrypt directamente. Esto permite cambiar el algoritmo sin modificar el código que lo consume, cumpliendo el principio de inversión de dependencias y aislando el mecanismo criptográfico.

Links

CamiloHenaoV/ProyectoPiedrazul

-- Fin del documento.

Referencias

Anexos

1. <https://docs.google.com/spreadsheets/d/1mAnrQiiUVUWu-FfYbVVNTRbaDRx6hNZ3DpW-x1-V3Y/edit?usp=sharing>

2. <https://www.figma.com/design/0TBfCOD3tlrNeGYASVDkCB/Wireframe-piedrazul?node-id=1140-6414&t=gD8UkpV5iaTShRHS-1>